m1

1. System Architecture Overview

This system is designed as a **decentralized educational runtime architecture** where the browser becomes the execution engine, and the PDF becomes the delivery mechanism.

Instead of traditional LMS (Learning Management Systems), the structure is composed of three core layers:

- **Distribution Layer (PDF Package)**
- **Decoding Layer (Browser Parser)**
- **Execution Layer (Runtime Engine)**

Each layer is fully self-contained and does not depend on external infrastructure.

https://www.udemy.com/course/web-based-3d-terrain-industrial-printing-engine-mastery/

2. PDF as a Self-Contained Application Container

In this architecture, the PDF is not a passive document.

It functions as:

- A **data container**
- A **code encryption layer**
- A **transport mechanism**
- A **lesson runtime trigger**

Inside the PDF, lesson logic is not stored as readable code. Instead, it is converted into **numeric character sequences**, which represent encoded HTML/JS/CSS structures.

This design allows:

- Offline portability
- Tamper resistance (logical integrity dependency)
- Single-file distribution
- Cross-device compatibility

When the PDF is uploaded into the browser environment, a fully local decoding process begins.

Step 1 — Local PDF Parsing

The browser reads the PDF using a client-side parser.

No network requests are made.

Step 2 — Numeric Stream Extraction

Only numeric sequences are extracted from the document's text layer.

These sequences represent encoded program logic.

Step 3 — Code Reconstruction

The numeric stream is grouped into structured character units and converted back into:

- HTML structure
- CSS styling rules
- JavaScript logic systems
- Simulation engine components

This rebuilds the full application in memory.

Step 4 — Runtime Deployment

The reconstructed system is executed inside a sandboxed environment using a Blob URL.

This ensures:

- No server deployment
- No persistent file creation
- No external dependency
- Session-only execution lifecycle

4. Offline Execution Model

The system is designed to function completely offline after download.

It does not require:

- User accounts
- Login systems
- Cloud authentication
- API validation

5. Procedural Terrain Engine Integration

Once decoded, the system launches a fully interactive procedural terrain engine.

This engine supports:

- Real-time terrain generation
- Noise-based landscape synthesis
- Hydrology simulation
- Erosion modeling
- Biome classification
- Water flow dynamics
- Terrain sculpting tools

The engine transforms theoretical algorithms into interactive simulation systems.

6. Simulation and Environmental Systems

The runtime engine includes layered environmental simulation models:

- Hydraulic erosion systems
- River path generation
- Basin detection logic
- Sediment transport simulation
- Terrain smoothing filters
- Biome distribution systems

These systems allow terrain to evolve dynamically based on algorithmic rules.

7. STL Export and Manufacturing Bridge

The system can export generated terrain as STL geometry.

This enables integration with:

- 3D printing slicers
- Manufacturing pipelines
- Physical model production systems

8. Core Architectural Philosophy

The system is built on three principles:

- **Decentralization** (no server dependency)
- **Portability** (single-file execution model)
- **Self-contained computation** (browser-only runtime)

It demonstrates that modern browsers can function as full-scale engineering environments capable of simulation, rendering, and digital fabrication workflows.

m2

© 2026 Okan Kaplan – MIT License Read full license